The Closed Machines

Eight kernel-checked conditional closures for BB(6) cryptids: what was proven, how, and the path to external approval

Enan Srivastava

September 3, 2026 (repository `Enan456/bbchallenge-proofs`)

The eight closures

Each machine below carries a theorem checked by the Lean 4 kernel at axioms {propext, Quot.sound}, with no `sorry`: *if its explicit arithmetic conjecture holds, the machine never halts from the blank tape.* Each reduction is complete (startup, every episode, and the transport to the abstract orbit are theorems) and, where tested, sharp (states violating the conjecture provably halt within tens of raw steps).

machine	keystone theorem	conjecture	lines
Antihydra	`safeOrbitA_nonhalting`	hydra orbit never dies	2,130
dossier-1	`dossier1_nonhalting`	chain orbit never dies	5,770
dossier-2	`dossier2_nonhalting`	`SafeMetaOrbit` (five integers)	6,130
dossier-3 (mirror)	`dossier3_nonhalting`	`SafeMetaOrbit` (mirrored)	9,020
Dvisetu (dossier-6)	`safeDvisetuOrbit_nonhalting`	`SafeCheckpointOrbit`	1,942
rank-1 (dossier-5)	`safeWordOrbit_nonhalting`	`SafeWordOrbit` (word orbit)	2,126
dossier-7	`safeD7Orbit_nonhalting`	`SafeD7Orbit` (word orbit)	1,036
dossier-8	`safeD8Orbit_nonhalting`	`SafeD8Orbit` (word orbit)	1,037

Machine codes: `1RB1RA_0LC1LE_1LD1LC_1LA0LB_1LF1RE_---0RA` (Antihydra), `1RB0LE_1LB1RC_0RF0RD_1RE0LA_1LA1RE_---0RB` (d1), `1RB1RC_1LC1LC_1RE0LD_1LB1RF_---1RA_0RD1RD` (d2), `1RB1RB_1LC0RE_---1LD_0RA1LB_1RA1LF_0LE1LE` (d3), `1RB0RB_1LB0LC_1RD1LC_1RE0RA_0RF1RA_---1LB` (Dvisetu), `1RB1RD_1LC0LC_0RD0LB_1RF0LE_0RD---_0RA1RC` (rank-1), `1RB1LA_1LC0RE_1LF1LD_1RE0LC_1LB1RA_---0LA` (d7), `1RB0LC_1LC0RD_1LF1LA_1LB1RE_1RB1LE_---0LE` (d8).

How each was closed

The method is uniform. **Watch**: sample the machine at a recurring configuration; the tape at those events has a bounded description (five integers for dossier-2/3, a hydra triple, one hole and two counters for Dvisetu, a word of blocks for rank-1). **Compress**: derive the exact law of that description empirically and fail-closed; every transition and its exact step count must be reproduced, any miss aborts. Evidence grades at freezing: dossier-2/3 validated to 10^9 raw steps; Dvisetu 2,746/2,746 timed transitions at 2×10^7; rank-1 2,179/2,186 frozen at 2×10^8 with the residue characterized, plus exact reseed differentials. **Certify**: compile the law into Lean: raw semantics, a halt-localization corridor lemma (each machine funnels every halt through one probe pattern), gait-level episode theorems with exact step counts, inductions for the unbounded middles, a concrete startup anchor, and a transport theorem showing the abstract orbit predicts the machine

forever. The conjecture is then a statement about an integer (or word) sequence, and the theorem converts it into non-halting.

Two engineering results from the newest closures: Dvisetu's one genuinely hard branch (the recursive divide renewal) is folded into its conjecture as a named, documented assumption rather than proven; rank-1, by contrast, needed *no* such fold: at cell-level grain its boundary families and decision function are subsumed and proven, making it the first closure on a state space of growing dimension. Dossiers 7 and 8 landed by the same pipeline taken to its logical end: their certified engines run at single-raw-step grain, so every branch of their record laws (including every residue case unobserved at the empirical horizon) is simulated exactly and proven faithful; no assumption is folded anywhere, and the only fail-closed exits are the two genuine halt patterns and fuel. Design mirrors validated at 2×10^7 raw steps (5,263/5,263 and 3,890/3,890 record transitions exact) before a line of Lean was written.

What it takes to push, validate, and get approved

1. External formal validation (Palomar). Prepared and pinned: five entries (four more to follow for Dvisetu, rank-1, and dossiers 7/8) with challenge/solution comparator pairs whose challenge files import Lean core only. Local intake: 0 failures, 0 warnings. *Remaining action: one browser session by the author* (GitHub authentication; the registry re-checks the proofs at the pinned commit and publishes the exact statements). Effort: under an hour. This satisfies the formal-proof-validation requirement of the community process.

2. Community publication and reproduction (bbchallenge.org). The community's adopted process: publish on the forum, invite reproduction, formal validation by a non-author, with an announcement-and-objection convention. A forum post is drafted (`docs/submissions/bbchallenge-forum-post.`) reduction framing, explicit non-claims (zero machines decided), and a one-command reproduction (`git checkout <pinned SHA> && lake build`). *Remaining actions: the author posts it; one independent person runs the build* (minutes of their time; the kernel does the validation). The scale criterion ($\geq$10% of undecided machines) applies to bulk deciders, not cryptid results; a sound invariant search over every learned law in the cohort returned zero invariant-decidable machines, so conditional reduction is the demonstrated frontier for this class.

3. Conference review (DeepMath 2026). The extended abstract (v18) is complete with the census, the six closures, the verification-game results, and honest scope; form fields are prepared. *Remaining actions: the author's OpenReview profile and submission click*; then peer review runs its course.

4. What approval cannot mean. No process turns these into unconditional results: each conjecture is a Collatz-class orbit avoidance statement, and the program's own negative results (no congruence invariant to modulus 384; ranking-class certificates exhaustively refuted on the true fatal model) show why. The honest end state of approval is: eight machine-checked reductions, externally re-verified, publicly reproduced, with each machine's halting problem restated as one explicit arithmetic question.

Verification for the reader.

```
git clone https://github.com/Enan456/bbchallenge-proofs
```

```
cd bbchallenge-proofs/busy-beaver-core && lake build
# per-theorem axiom reports are printed by the build itself
```